



Aeroflutter

Employee Attendance App — How We Built It

A complete, non-technical walkthrough of every decision, every error, and every fix — from blank screen to working APK.

Prepared for: Rahul Bhure | Date: May 2026



Table of Contents

1. What is Aeroflutter? What did we build?
2. The Building Blocks — Tools & Technologies (explained simply)
3. The Database — Where data lives
4. Phase 1 — The Login & Signup Screens
5. Phase 2 — GPS at Signup
6. Phase 3 — Face Registration
7. Phase 4 — Daily Check-In / Check-Out
8. All the Errors We Faced & How We Fixed Them
9. Getting the App on Your Phone — The APK Build
10. Summary & What's Next

1. What is Aeroflutter? What Did We Build?

Aeroflutter is a **smart attendance tracking system** built for a company that wants to make sure employees are physically present at the office before they can mark their attendance. It has two parts:

Part	Who uses it	What it does
 Mobile App	Employees	Sign up, register face, check in/out daily using GPS + face scan
 Web Dashboard	Owner / HR	See all attendance records, employee list, reports

This document covers the **mobile app** journey from zero to a working APK file.

 What makes this app special?

A normal attendance app just asks you to click "Check In." Aeroflutter verifies *three things* before accepting:

- 1. **Your identity**
 - Face scan matches your registered face
- 2. **Your location**
 - You must be within 50 metres of the office
- 3. **Your account**
 - You must be logged in with your company email

2. The Building Blocks — Tools & Technologies

Before we wrote a single line of code, we chose the tools. Think of this like choosing materials before building a house.

 **Simple Analogy:** *Building a mobile app is like building a house. You need bricks (React Native), a blueprint language (JavaScript), plumbing plans (Supabase for data), and a construction company (Expo) to put it all together.*

Tool	What it is (simply)	Why we used it
------	---------------------	----------------

React Native	A framework that lets you write one codebase for both Android and iPhone	Write once, works on both platforms
Expo SDK 49	A helper platform built on top of React Native — gives you camera, GPS etc. ready-made	Saves weeks of setup work
JavaScript	The programming language used — like English is the language of business	Most widely used, fast to develop
Supabase	An online database + login system — like a smart Excel sheet in the cloud	Free, fast to set up, handles login security automatically
React Navigation	Controls which screen shows when — like a table of contents for the app	Industry standard for React Native apps
expo-location	Accesses the phone's GPS	Needed to verify office proximity
expo-camera	Accesses the phone's camera	Needed for face registration and check-in
expo-face-detector	Google's AI that finds faces in a camera frame	Detects face landmarks for comparison
Node.js	The engine that runs JavaScript on your computer (not just in a browser)	Required to run the development server
EAS Build	Expo's cloud service that compiles your code into an installable APK	Builds APK without needing Android Studio

3. The Database — Where All Data Lives

 **Simple Analogy:** *The database is like a filing cabinet. Instead of paper files, it stores digital information — employee names, GPS coordinates, face data, and attendance records — all securely in the cloud.*

We used **Supabase** as our database. It's hosted online, so data is safe even if your computer crashes. We created four tables (think of each as a separate drawer in the filing cabinet):

Table Name	What it stores
employees	Name, email, phone, office GPS coordinates captured at signup, face encoding data, whether onboarding is complete
attendance	Each check-in and check-out event — employee ID, time, GPS location at check-in/out, status
audit_logs	A detailed history of every action — including failed attempts, with reasons
offices	Office locations (for future multi-office support)



Security Rules (RLS — Row Level Security)

We set up rules so that employees can only see their OWN data. An employee cannot see another employee's attendance records. Only the owner/admin can see everything. This is like each employee having their own locked drawer.

4. Phase 1 — Login, Signup & Auth Screens

The first screens we built were the "front door" of the app — the screens you see before you're logged in.



Screens Built in Phase 1

Screen	Purpose
Login Screen	Employee enters email + password to access the app
Signup Screen	New employee creates an account (name, email, phone, password)
Forgot Password Screen	Sends a reset link to the employee's email
Email Verification Screen	Shown after signup to remind user to verify their email



How login security works

When you type your password, Supabase checks it against its secure database. It never stores the actual password — it stores an encrypted version (called a hash). Even if someone hacked the database, they couldn't read passwords.

We also built an **AuthContext** — think of this as a central "memory" for the app. Once you log in, the app remembers your session so you don't have to log in again every time you open it.

5. Phase 2 — GPS Capture at Signup

 **Simple Analogy:** *Imagine every new employee, on their first day, stands inside the office and the app takes a "GPS photo" of exactly where they are. This becomes their office anchor point forever. When they check in later, the app compares their current location to this saved point.*

We added a critical feature to the Signup Screen: **before submitting the form, the app captures the employee's GPS location.** This GPS point becomes their permanent "office location" — stored as latitude and longitude in the database.

Important Rule We Enforced

Employees must sign up *from inside the office*. Their sign-up GPS coordinates become the reference point for all future check-ins. If they sign up from home, check-in will never work for them.

How GPS verification works at check-in:

We used the **Haversine Formula** — a mathematical calculation that computes the real-world distance (in metres) between two GPS coordinates, accounting for the Earth's curvature.

The threshold is **50 metres**. If you're within 50m of your signup location, you're considered "at the office." If not, the Check-In button is disabled.

6. Phase 3 — Face Registration (Onboarding)

💡 **Simple Analogy:** *Think of this like a fingerprint scanner at a door. The first time, you press your finger and the scanner "learns" your fingerprint. Every time after, it compares your finger to what it learned. We did the same thing with faces — using the front camera instead of a fingerprint reader.*

After an employee logs in for the first time, they are taken to the **Face Registration screen** before they can use the app. This happens only once.

👉 How Face Registration Works — Step by Step

- 1 The app opens the front camera
- 2 Google's on-device AI (expo-face-detector / ML Kit) scans the camera 2 times per second, looking for a face
- 3 When a face is found, it identifies **8 key landmarks**: left eye, right eye, nose tip, left mouth corner, right mouth corner, left cheek, right cheek, bottom of mouth
- 4 These 8 landmark positions are **normalized** — converted to ratios relative to the face size, so they work regardless of how close or far you are from the camera
- 5 The 8 normalized coordinate pairs are saved to the Supabase database as the employee's "face encoding"
- 6 The employee's record is marked as `is_onboarded = true` — they won't see this screen again

💡 How Face Comparison Works at Check-In

At check-in, the app captures a fresh face scan and compares the 8 landmark positions to the stored ones. It calculates the "distance" between the two sets of points — if the difference is less than 0.25 (our threshold), it's a match. The comparison happens entirely on the device — no photo is sent anywhere.

7. Phase 4 — Daily Check-In / Check-Out



The Daily Attendance Flow

Every working day, employees go through this flow:

- 1 Home Screen** — Shows live distance from office (updates as you walk), today's check-in/out times, and status
- 2 Check-In Button** — Only enabled when you are within 50 metres of the office. Tapping it starts the verification flow.
- 3 GPS Check** — App gets a fresh GPS reading. If more than 50m away, it stops and shows "Too Far from Office"
- 4 Face Scan** — Front camera opens, scans your face, compares to stored encoding
- 5 Attendance Recorded** — Check-in time and GPS coordinates saved to the attendance table in Supabase
- 6 Audit Log** — Every attempt (success or failure) is logged with the reason, for the owner's reference

Other screens built in Phase 4:

Screen	What it does
Check-Out Screen	Same GPS + face flow, but records departure time
History Screen	Shows last 30 days of attendance — present days with times, absent days highlighted
Profile Screen	Shows employee's name, email, office location status, face registration status

8. All the Errors We Faced & How We Fixed Them

Building an app never goes smoothly in one shot. Here is every significant error we encountered, explained simply, and how we solved it.

✖ Error 1: PowerShell blocked npx from running

What happened: When we tried to start the development server by typing `npx expo start` in the terminal, Windows showed: *"File cannot be loaded because running scripts is disabled on this system."*

Why it happened: Windows PowerShell has a security setting that blocks .ps1 script files (a type of program file) from running. The `npx` command uses such a file.

💡 **Simple Analogy:** *Windows is like a security guard who won't let certain staff into the building. We found a back door — a different version of the same command that the guard allows.*

✅ **Fix:** We changed `npx` to `npx.cmd` — the `.cmd` version is a batch file (not a .ps1 script), which Windows allows freely.

❌ Error 2: ERR_PACKAGE_PATH_NOT_EXPORTED (Metro internal paths blocked)

What happened: When we ran the server on Node.js version 24, we got an error about *"package path not exported"* for Metro (the tool that bundles the app code).

Why it happened: Node.js version 24 became strict about package access rules. Metro (version 0.84.4) has internal folders it uses, but didn't officially "announce" those folders in its configuration file. Node 24 enforced this rule strictly and blocked access.

💡 **Simple Analogy:** *Imagine an office building where employees need keycards to access different floors. Metro had an internal staircase it used, but never listed it in the official building directory. New strict security (Node 24) found it wasn't listed and locked the door.*

✅ **Fix:** We edited Metro's configuration file to officially add those internal paths to its "allowed access" list. We patched the file `node_modules/metro/package.json` to include `"./src/*":
"./src/*.js"`.

❌ Error 3: MetroHmrServer is not a constructor (Node 24)

What happened: After fixing Error 2, a new error appeared: *"MetroHmrServer is not a constructor."* This happened when using Node.js version 24.

Why it happened: Metro's "Hot Module Replacement" server (which lets the app update instantly when you change code) was written using a newer JavaScript export style. But the code that imports it expected the old style. Node 24 changed how this export/import works, causing a mismatch.

💡 **Simple Analogy:** *It's like ordering a product and receiving the box inside-out — everything is there, but you can't open it the normal way.*

✅ **Fix:** We switched from Node.js 24 to Node.js 18 (using a tool called nvm-windows). Expo SDK 49 was designed for Node 16–18.

❌ Error 4: `configs.toReversed` is not a function (Node 18)

What happened: After switching to Node 18, a different error appeared: "`configs.toReversed` is not a function."

Why it happened: The Metro config loader used a JavaScript array feature called `toReversed()`, which was only added in Node.js version 20. Node 18 doesn't have it.

💡 **Simple Analogy:** *It's like asking someone who learned English in 2015 to use slang that was invented in 2022 — they simply don't know that word exists yet.*

✅ **Fix:** We directly edited the Metro config loader file (`metro-config/src/loadConfig.js`) and replaced `configs.toReversed()` with the equivalent older code: `[...configs].reverse()` — which does the same thing but works in Node 18.

❌ Error 5: `npm install` wiped our patches

What happened: At one point, the user ran a command that reinstalled all packages. This deleted all our manual fixes to the Metro files.

Why it happened: `npm install` downloads fresh copies of all packages from the internet, completely replacing any changes we had made manually.

💡 **Simple Analogy:** *Imagine you repaired a borrowed book by taping some pages. Then the library recalled it and gave you a fresh unrepaired copy.*

✅ **Fix:** We re-applied all the patches (metro/package.json exports fix + loadConfig.js toReversed fix) from scratch. We also noted this as a fragile point.

❌ Error 6: MetroHmrServer is not a constructor (Node 18, different cause)

What happened: Even on Node 18, after all the fixes above, a new version of the "MetroHmrServer" error appeared.

Why it happened: Metro's HmrServer module (which handles live code updates) exports its main class using the modern JavaScript "default export" style. But the Expo CLI code that imports it was written expecting the older "direct export" style — it got the wrapper box instead of the class inside it.

💡 **Simple Analogy:** *The module handed over a gift-wrapped box, but the receiver expected the gift directly. They didn't know to unwrap it.*

✅ **Fix:** We patched Expo CLI's `resolveFromProject.js` file for both `importMetroHmrServerFromProject` and `importMetroCreateWebsocketServerFromProject` to add the ".default" unwrapping step: `mod.__esModule && mod.default ? mod.default : mod`

❌ Error 7: "Tailwind CSS not configured with NativeWind preset"

What happened: When we set up NativeWind (a styling library), we got an error about the preset not being configured.

Why it happened: NativeWind version 4 requires a special "preset" line in the Tailwind config file. We had missed adding it.

✅ **Fix:** Added `presets: [require('nativewind/preset')]` to `tailwind.config.js` .

✖ Error 8: "Something went wrong" on Expo Go (NativeWind v4 incompatibility)

What happened: When scanning the QR code on the phone, Expo Go showed "Something went wrong" and crashed.

Why it happened: NativeWind version 4 (a CSS styling system for React Native) requires React Native 0.73 or newer. Our app uses React Native 0.72 (Expo SDK 49). NativeWind v4's internal modules were not compatible.

💡 **Simple Analogy:** *It's like installing a 2024 car part into a 2022 car — the part is too new and doesn't physically fit the mounting points.*

✅ **Fix:** We completely removed NativeWind from the build pipeline — changed `babel.config.js` back to plain Expo preset, removed `withNativeWind` from `metro.config.js`, and removed the CSS import from `App.js`. Since all our screens use React Native's built-in styling (not NativeWind's CSS classes), nothing visual changed.

✖ Error 9: Wrong import paths in screen files

What happened: Several screens (Login, Forgot Password, Email Verification) were importing from the wrong folder path.

Why it happened: The screens are inside `src/screens/auth/` — two levels deep. The imports were written as if they were only one level deep.

💡 **Simple Analogy:** *Like giving someone directions to your house starting from the wrong street — they're close but they'll end up at the wrong door.*

✅ **Fix:** Changed all imports from `'../context/AuthContext'` to `'../../context/AuthContext'` — one extra `../` to go up one more folder level.

✖ Error 10: `signup_latitude / signup_longitude` NOT NULL constraint

What happened: The original signup code would crash with a database error because it wasn't providing the GPS coordinates when creating a new employee record.

Why it happened: The database was designed to require GPS coordinates for every employee (NOT NULL constraint). But the original signup function didn't collect GPS at all.

✅ **Fix:** We redesigned the Signup Screen to capture GPS location *before* the form is submitted, and pass those coordinates into the employee creation function.

9. Getting the App on Your Phone — Building the APK

💡 **Simple Analogy:** *All the code we wrote is like a recipe. An APK file is the finished cooked meal — ready to eat (install) on your phone. The EAS Build service is like a professional kitchen that cooks the recipe for us without us needing any cooking equipment (Android Studio).*

📦 Steps to Build the APK

- 1 **Created eas.json** — a configuration file that tells Expo's build service: "build an APK file for Android"
- 2 **Installed EAS CLI** — a command-line tool that talks to Expo's servers (`npm install -g eas-cli`)
- 3 **Logged in to Expo account** — account: rwbhure07 at expo.dev
- 4 **Generated Android Keystore** — a digital "signature" file that proves the APK came from you. Like a wax seal on a letter.
- 5 **Uploaded code to EAS Build** — 175 KB of compressed project files sent to Expo's cloud servers
- 6 **Build running on Expo's servers** — takes ~15 minutes. No laptop needed — can close computer.
Build URL: `https://expo.dev/accounts/rwbhure07/projects/aeroflutter-employee/builds/41ef5aa7...`
- 7 **Download & Install** — once done, download the .apk file from the Expo dashboard and install it on your phone

📧 **Email notification**

Expo will send an email to rahulbhure1996@gmail.com when the build is complete with a download link.

How to install the APK on Android:

1. Open the download link on your Android phone's browser
2. Download the `.apk` file
3. Go to **Settings** → **Security** → **Install unknown apps**
4. Allow your browser to install apps
5. Tap the downloaded file and press **Install**

10. Complete List of Files Created/Modified

Here is every file we created or changed, and what it does in one sentence:

📁 Configuration Files

`app.json`

Main app config — name, version, permissions for camera and location, Android package ID

`eas.json`

Tells EAS Build to create an APK file for Android (not a Play Store bundle)

`babel.config.js`

JavaScript compiler settings — tells the build tool how to translate modern JS into code phones understand

`metro.config.js`

Metro bundler settings — includes Node 18 compatibility polyfills

📁 Database & Auth

`src/config/supabase.js`

Connection to the Supabase database — URL and secret key

`src/context/AuthContext.js`

The app's "memory" — stores login state, employee profile, onboarding status; provides login/signup/logout functions to all screens

Navigation

`src/navigation/RootNavigator.js`

The "traffic controller" — decides which section of the app to show based on login and onboarding state

`src/navigation/AuthNavigator.js`

Groups all pre-login screens together

`src/navigation/OnboardingNavigator.js`

Shows face registration screen for first-time users

`src/navigation/AppNavigator.js`

Groups all main app screens (Home, CheckIn, CheckOut, History, Profile)

Screens

`src/screens/auth/LoginScreen.js`

Email + password login form

`src/screens/auth/SignupScreen.js`

New account form — captures GPS location before submission

`src/screens/auth/ForgotPasswordScreen.js`

Sends password reset email via Supabase

`src/screens/auth/EmailVerificationScreen.js`

Tells user to check their email after signup

`src/screens/onboarding/FaceRegistrationScreen.js`

Opens front camera, detects face landmarks, saves face encoding to database

`src/screens/app/HomeScreen.js`

Main dashboard — shows live distance from office, today's attendance, check-in/out buttons

`src/screens/app/CheckInScreen.js`

Full GPS + face verification flow for morning check-in

`src/screens/app/CheckOutScreen.js`

Full GPS + face verification flow for end-of-day check-out

`src/screens/app/HistoryScreen.js`

30-day attendance calendar with present/absent markers and check-in/out times

`src/screens/app/ProfileScreen.js`

Shows employee's profile info and onboarding status

Services (Database functions)

`src/services/employees.js`

Functions to get employee profile, save face encoding, update GPS location

`src/services/attendance.js`

Functions to check in, check out, get today's record, get 30-day history, write audit logs

Utilities (Helper functions)

`src/utils/geofence.js`

Haversine distance calculator — computes metres between two GPS points

`src/utils/faceComparison.js`

Normalizes face landmarks and compares two faces mathematically

 Context (Shared live data)

`src/context/LocationContext.js`

Continuously tracks GPS and calculates distance from office — all screens can read this

11. Summary — The Big Picture

What we built	Status
Database schema + security rules in Supabase	✓ Done
Login, Signup, Forgot Password screens	✓ Done
GPS capture at signup	✓ Done
Face registration onboarding	✓ Done
Daily check-in with GPS + face verification	✓ Done
Daily check-out with GPS + face verification	✓ Done
30-day attendance history screen	✓ Done
Profile screen	✓ Done
Audit logging for all check-in/out attempts	✓ Done
APK build submitted to EAS Build	🕒 Building

**What makes this app production-ready:**

- No fake check-ins — requires physical presence (GPS) + identity (face)
- All data is private — employees see only their own records
- Every attempt is logged — full audit trail for disputes
- Works offline for face comparison — no privacy concerns, no photo sent to server
- Scalable — adding more employees or offices requires no code changes

12. What's Next

-  Install the APK and do full end-to-end testing on a real phone
-  Build the **Owner Web Dashboard** (React + Vite) — see all employees, daily attendance, export reports
-  Add push notifications for missed check-ins
-  Add multi-office support
-  Publish to Google Play Store (for easier distribution)

Aeroflutter Employee App • Built with Expo SDK 49 + React Native + Supabase

Document prepared by Claude AI | May 2026